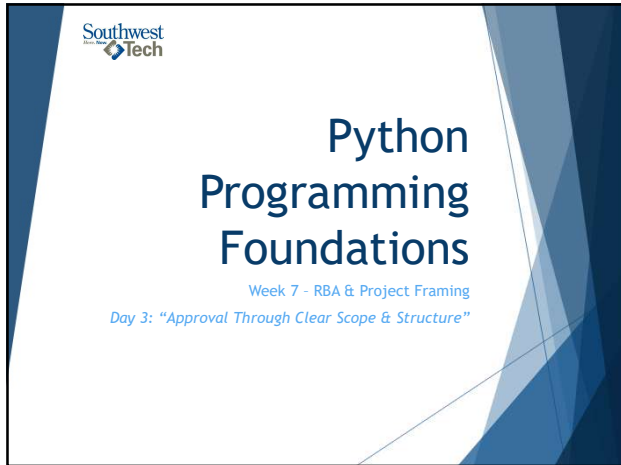
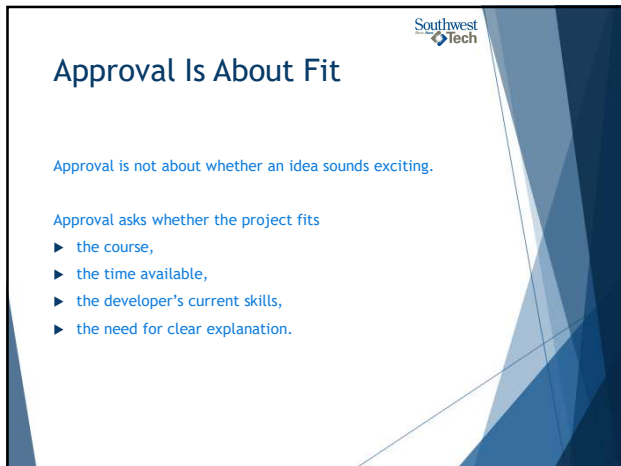




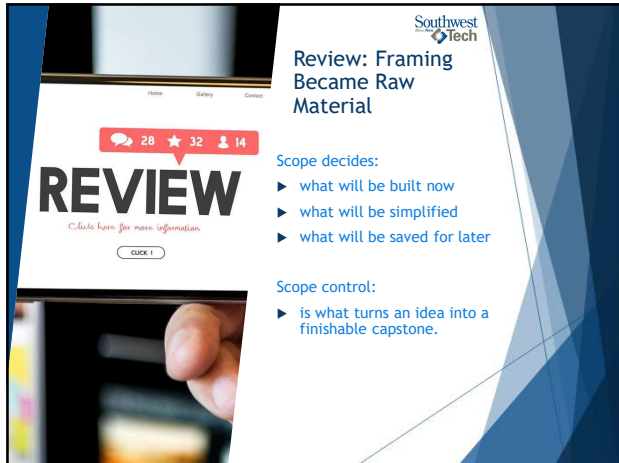
1



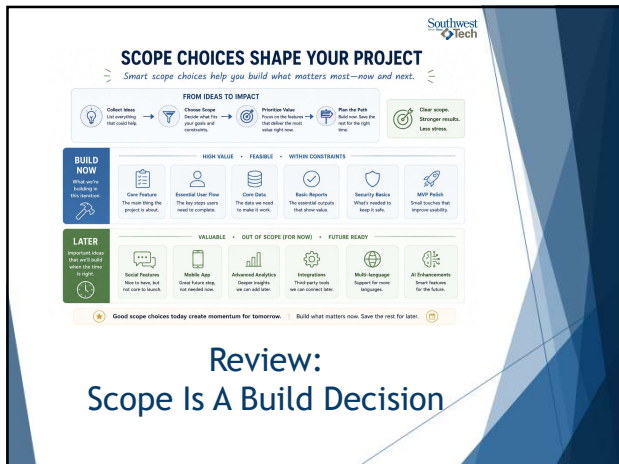
2



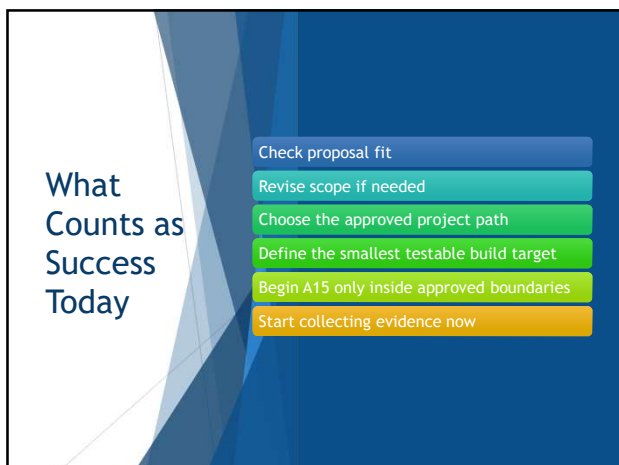
3



4



5



6



Today's Success

7

What We Will Use Today

- ▶ approval criteria
- ▶ project path comparison
- ▶ first build target
- ▶ run instructions
- ▶ validation evidence
- ▶ AI accountability

8

TODAY'S LAUNCH TOOLS
Use these tools to launch your capstone with clarity and confidence.

- 1. APPROVAL CRITERIA**
 - Pay close attention
 - Research in your area
 - Match the audience
 - Confirm its feasibility
- 2. PROJECT PATH**
 - Target
 - Timeline
 - Task setting
 - Transfer insight
 - Clear build path
- 3. FIRST BUILD TARGET**
 - Define a small, specific target that creates real progress
 - Concrete target
 - Visible action
 - Transfer insight
 - Reason to build it first
- 4. RUN INSTRUCTIONS**
 - Build within the approved focus, structure, and focus boundaries
 - In-scope only
 - At one step forward
 - Change, extend and improve
 - Build on one step
- 5. VALIDATION EVIDENCE**
 - Collect evidence from day one. It proves your work and your thinking
 - Track and update
 - Explanations
 - Structure and changes
 - At one step
- 6. AI ACCOUNTABILITY**
 - Follow the pattern. Build with focus. Finish with confidence
 - Small steps. Clear choices. Strong results

Toolbox

9

Southwest
Tech

What We Will “Skip” For Now

- ▶ final polish
- ▶ presentation rehearsal
- ▶ optional features
- ▶ unapproved expansions
- ▶ professional packaging

Today we approve the scope and start the smallest useful build.



10

Southwest
Tech

PARKED FOR LATER

*Important, but not today.
We will return to these after the core is done.*

FINAL POLISH	PRESENTATION REHEARSAL	OPTIONAL FEATURES	UNAPPROVED EXPANSION	PROFESSIONAL PACKAGING
Refine styles, spacing, and small details to make it shine.	Practice the demo and talking points for confidence.	Nice to have ideas that can be added after the core.	New directions or big changes that are out of scope.	README, branding, or advanced docs are for later.

PARK IT HERE.
Focus now. Finish strong.

TODAY: APPROVE SCOPE AND START THE CORE.

11

Southwest
Tech

Project Approval/ Revision



12

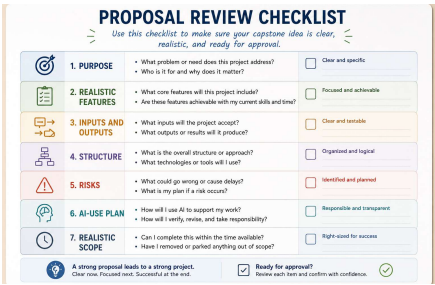


Approval Criteria

An approvable proposal shows:

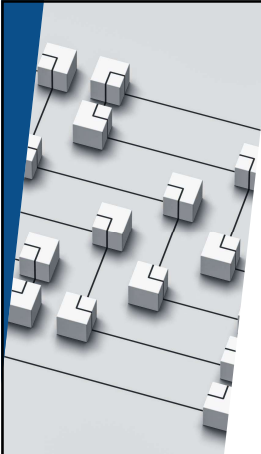
- ▶ clear purpose
- ▶ realistic features
- ▶ expected inputs and outputs
- ▶ likely structure
- ▶ risks or unknowns
- ▶ AI-use plan
- ▶ why the scope is realistic

13



Approval Criteria

14



Project Path Changes Complexity

Possible capstone paths:

- ▶ console utility
- ▶ data-driven tool
- ▶ file-based tracker
- ▶ API-style fetcher
- ▶ tight architecture-preview project, if approved

Each path carries different risks.

15

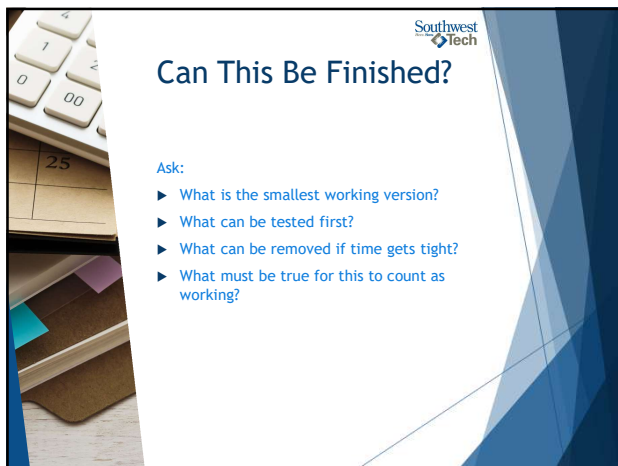
PROJECT PATH COMPARISON

Five approved paths. Choose the one that fits your data, skills, and time.

	1. CONSOLIDATE UTILITY	2. DATA-DRIVEN TOOL	3. FILE TRACKER	4. API FETCHER	5. ARCHITECTURE REVIEWER
What it is	A simple get-go case in the console and solves a specific task.	A tool that works with data in files, docs, or simple structures.	A cloud-first tool, spins, and ingests data in local files.	A tool that gets data from an external API and shows simple results.	A small project that takes the role of a target applet.
Example	Budget calculator with name	Course analyzer from a data file	Notes tracker with one-based	Master's ledger tool	Printer of a full application flow
Core case	Logic, input handling, program flow	Data handling, calculation, program flow	File I/O, persistence, pagination	API calls, data parsing, handling	Structure, components, handling
Typical skills used	Variables, conditionals, loops, functions, hashset	Lists, dictionaries, lists, functions, converting data	File operations, data validation, simple runtime	Requests, JSON, data mapping, basic error handling	Modular design, lists, interfaces, class flow
Find target	A working menu with one useful feature	Load data and show one meaningful report	Add one item and save it successfully	Fetch data and display it clearly	Show the components and how they connect
Wish to reach	- Simple menu - One handy feature - Wish input validation	- Customizable data - Need to explore logs - Missing input checks	- File format issues - Data race - Poor error handling	- API status or changes - Network errors - Online data mapping	- No real design - No handling needed - Need to explore value

Any path can lead to a strong capsule. Pick the one that fits, keep it focused, and build over purpose.

Choose your path. Define your first target. Take the first step today.



Can This Be Finished?

Ask:

- ▶ What is the smallest working version?
- ▶ What can be tested first?
- ▶ What can be removed if time gets tight?
- ▶ What must be true for this to count as working?

Southwest
Tech

FINISHABILITY: BUILD WHAT YOU CAN FINISH

"A clear boundary today helps you deliver a working project."

SMALL, TESTABLE CORE

Build this now

- Code that solves the problem
- Doesn't have an escape route
- Clear intent or goal
- Measures up to end

Goal: Probe the idea early with a simple, executable prototype.

LATER FEATURES

Save these for after the core works.

- Advanced options and settings
- Reports and visualizations
- Check any or no external integrations
- Rich, dynamic, and extra content

Goal: Add value and polish after the core is stable and complete.

↔

CHECK YOUR PLAN

- #### 1. SMALLEST WORKING VERSION?

What is the smallest thing I can build that will prove the idea works?
- #### 2. TEST FIRST?

Can I test this core early to confirm it works before adding more?
- #### 3. REMOVE IF TIGHT?

What features can I remove or delay if time or scope gets tight?
- #### 4. COUNTS AS WORKING?

Does the core deliver real value and solve the main problem for the user?

1

You should have to build everything. Establish a focused core. Then make it even better.

2

Finishability is a feature. A focused core is better than a perfect plan.

Can This Be Finished?



Can This Be Explained?

Ask:

- ▶ Can I describe the main logic?
- ▶ Can I explain the data?
- ▶ Can I show validation evidence?
- ▶ Can I explain AI contributions, if any?

19



EXPLAINABILITY CHECK

Before you submit, make sure you can explain your project clearly.

1. MAIN LOGIC	2. DATA	3. VALIDATION EVIDENCE	4. AI CONTRIBUTION
- What problem does it solve? - How does the logic work? - What happens step by step? Example: Calculates total using a loop and conditions.	- What data does it use? - Where does it come from? - How is it organized? Example: Reads items from a file and stores in a list.	- What did you test? - What is the output or result? - How do you know it works? Example: Tested with 3 cases and got the expected results.	- What did AI help you with? - What did you change? - What did you learn? Example: AI helped with code structure. I wrote the logic.

MY EXPLANATION
I can explain how my program works, what data it uses, how I tested it, and how I used AI responsibly.

★ If you can explain it in your own words, you're ready. ✓ Clear thinking. Clear project. Confident submission.

Can This Be Explained?

20

Revision Is Not Rejection

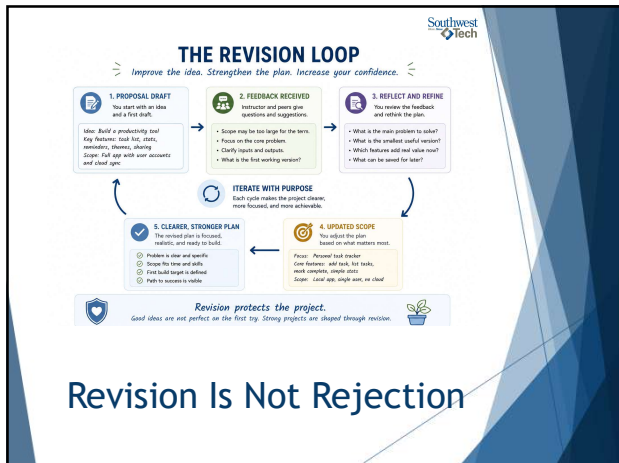


If the proposal needs revision, that is not failure.



Targeted revision protects the student from building something too large, unclear, or hard to explain.

21



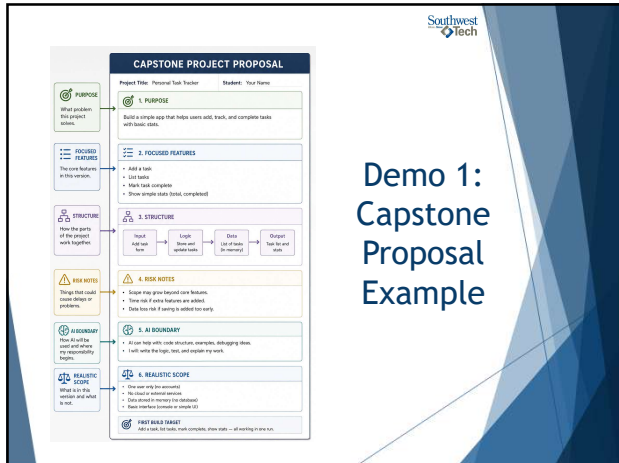
22



23



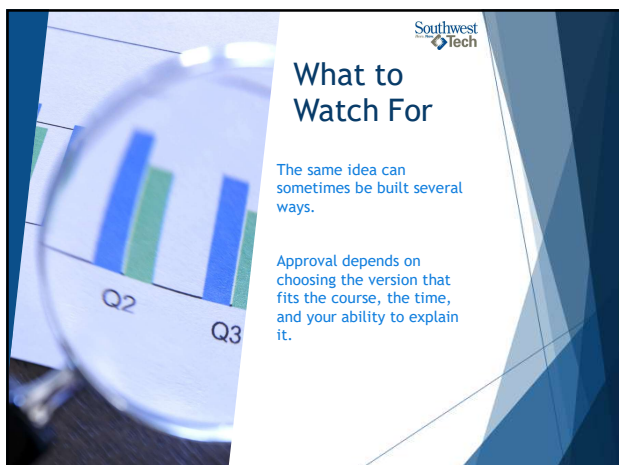
24



25



26



27

STRUCTURE FIT: SAME IDEA, SEVERAL PATHS

One idea can be built in different ways. Approval selects the path that fits your time, your course, and your ability to explain it clearly.

Core Idea (same in every path): Personal task tracker — add tasks, mark complete, show simple stats.				
A. COMBODU UTILITY	B. DATA-DRIVEN TOOL	C. FILE TRACKER	D. API FETCHER	E. ARCHITECTURE READER
Use keywords in the complex program process type and show results.	Data stored in tables; presented in memory; results displayed.	Data used to track items to find files from other interfaces.	Program requests data from an external API and displays useful results.	Show how programs present, design or simple modules.
Data handled: Manual in memory only.	In memory (local/DB).	In local file (e.g., JSON).	From external service (API required).	Example data or mock data.
Complexity: Lower.	Medium.	Medium.	Medium.	Varies (focus on design).
Time to first working system: Fast.	Medium.	Medium.	Medium.	Medium.
Explaining the tool: Easy to explain step by step.	Easy with clear idea flow.	Easy with clear idea flow.	Easy if API calls are clear and results are shown.	Easy other components and connections are clear.
What you learn: Logic, heap, index, conditions, loops.	Data structures, operations, simple reporting.	File I/O, persistence, validation.	API, data formats, error handling.	Structure, components, data flow, design thinking.
APPROVAL SELECTS THE PATH THAT FITS	YOUR TIME How long can you build and at the time you want?	YOUR COURSE How does your idea fit with what you are learning and will be approved?	YOUR COMPLEXITY How complex is your idea? How hard to build?	APPROVED IDEAS You built the version that fits the time, the data, and can be explained with confidence.

Different paths. Same goal. The right choice is the one you can complete and explain.

Demo 2: Structure Options & Approval Fit

28

Assignment #14 Checkpoint

For A14, submit or revise the proposal until the approved scope is clear.

Do not begin major build work outside the approved or provisionally approved scope.

29

CHECKPOINT: WHERE IS YOUR PROPOSAL?

This checkpoint helps you know your next step.

1. APPROVED

Your proposal is clear, focused, and ready to move forward.

What this means:

- Purpose is clear
- Features are focused and realistic
- Structure fits the idea
- Risks are understood
- All boundary is defined
- Scope fits time and course

Next Step: Start building the core. Test early and keep it simple.

2. PROVISIONALLY APPROVED

The idea is strong, but a few areas need adjustment.

What this means:

- Most parts are clear
- Some features may be too large
- Structure needs small changes
- Risks or scope need more detail
- All use or data plan needs clarity

Next Step: Make the needed adjustments. Then move to final approval.

3. REVISE BEFORE MAJOR BUILD

Key parts are missing or unclear. More work is needed before building.

What this means:

- Purpose is unclear or too broad
- Features are too many or not focused
- Structure is missing or unclear
- Risks or scope are not realistic
- All boundary is missing or vague

Next Step: Revise your proposal. Then return for review.

This is not about right or wrong. This checkpoint helps you choose the best next step for a successful project.

Assignment 14 Checkpoint

30



Assignment #15

Once your project is approved or provisionally approved, begin Assignment 15.

Start with the smallest testable core, not the full imagined project

31



FROM APPROVED PROPOSAL TO FIRST BUILD TARGET

Start small. Test early. Grow with confidence.

APPROVED PROPOSAL

Your proposal has been reviewed and approved.

Key Elements Confirmed:

- ✓ Purpose
- ✓ Focused Features
- ✓ Structure
- ✓ Risk scope
- ✓ Accessibility
- ✓ Analytic scope

You have a clear plan and a shared understanding.

APPROVED SCOPE

This is the version you will build first. It fits your time, scope, and goals.

Included in This Version:

- Core features only
- Simple, clear structure
- Limit data (if any)
- Single user
- Testable end-to-end

Everything else is saved for future versions.

SMALLEST TESTABLE CORE

Build the minimum that proves the idea works.

First build includes:

- Add item
- List items
- Mark complete
- Show sample data

Evidence You Will Produce:

- Working program
- Test results
- Short explanation

ATIS BEGINS

You build the smallest testable core, gather evidence, and explain your work.

NOT EVERYTHING TODAY

Additional features, improvements, and actions are planned for later versions.

Progress comes from steady steps. Focus on what matters now. Build, test, learn, improve.

32



First Build Target



Your first build target should answer:

- ▶ What file or function starts the project?
- ▶ What input can I test?
- ▶ What output proves progress?
- ▶ What will I validate first?

33



FIRST-BUILD PLANNING

A few clear answers today make your first build simple and successful.

Approved Scope → First Build Target (Smallest testable core) → A1S Begins (Build, test, and learn)

PLAN YOUR FIRST STEP

1. WHAT FILE OR FUNCTION STARTS?

Choose the first place of code you will create.

Example: main.py or add_task() function

2. WHAT INPUT CAN I TEST?

Pick a simple, realistic input you can try right away.

Example: A task name typed in "Buy groceries"

3. WHAT OUTPUT PROVES PROGRESS?

Decide what you will see that shows it's working.

Example: Task appears in the list
Total tasks = 1

4. WHAT VALIDATION FIRST?

Choose one basic check to keep things visible.

Example: Task name is not empty
No duplicates allowed

KEEP IT SMALL

One simple step. One clear test. One visible result. That is progress.

YOU CAN GROW IT LATER

Extra features, improvements, and polish come after the core works.

First Build Target

34



Evidence To Start Collecting Now



Start collecting evidence now:

- ▶ project files
- ▶ run instructions
- ▶ sample input/output
- ▶ validation notes
- ▶ change or revision notes
- ▶ AI-use notes, if used

35



PROJECT FOLDER EVIDENCE

Keep your work organized. Make your thinking visible.

1. PROJECT FILES

All files needed to run and explain your project.

```
my_project/
├── src/
│   ├── main.py
│   ├── task.py
│   └── ...
├── data/
│   ├── tasks.json
│   ├── test_data.py
│   └── ...
├── README.md
├── requirements.txt
└── ...
```

Tip: Keep files small, clear, and well-named.

2. RUN NOTES

How you ran it and what happened.

Date: June 14, 2020

Command: python main.py

Environment: Python 3.7.3

Notes: Program started correctly. No errors.

3. SAMPLE INPUT / OUTPUT

A small example that shows it works.

Sample Input:

```
Buy groceries
Add task
Read book
```

Sample Output:

```
1. Buy groceries
2. Add task
3. Read book
Total tasks: 3
```

4. VALIDATION NOTES

What you checked and why it matters.

- Added a task → task shows in list
- Empty list → shows error message
- No duplicate → duplicate rejected
- Input updates correctly
- Improved total count display

Why it matters: These checks show the core works and is stable.

5. REVISION NOTES

Changes you made and why.

- AT1: Simplified task format to text
- AT2: Fixed task format to be consistent
- AT3: Fixed duplicate check to be case insensitive
- AT4: Improved total count display

Why: These changes make it clearer, safer, and easier to use.

6. AI-USE NOTES

How AI was used in this project.

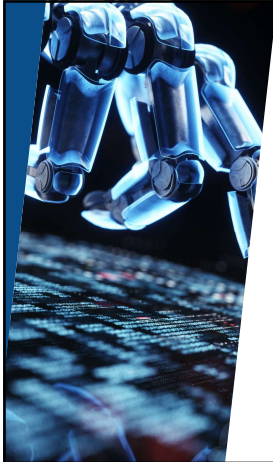
- AI was used to:
 - Research feature ideas
 - Suggest error handling ideas
 - Explain a Python function

AI did not write the core logic, comments, and other all suggestions.

Your goal: Make it easy for someone (including your future self) to understand, run, and trust your work.

Evidence To Start Collecting Now

36

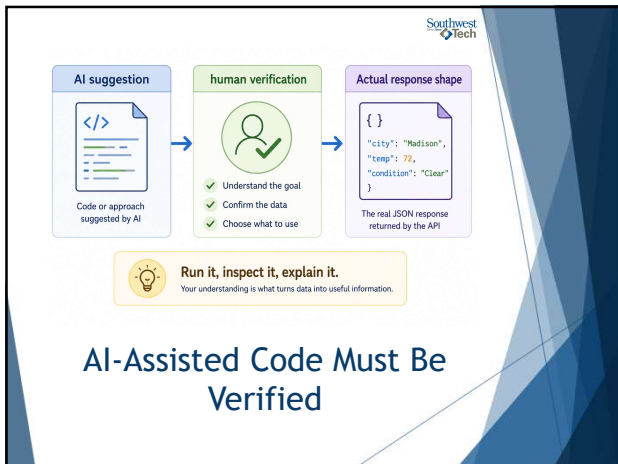


Southwest
Tech

AI Use

- ▶ AI can suggest API code that looks believable but does not match the actual response.
- ▶ You must inspect the response, run the code, and explain what human decision you made.

37



38



Southwest
Tech

Closing Success Check

By the end of today, you should be able to say:

"My project scope is approved or being revised, and I know the smallest useful piece to build first."

39

Southwest
Tech

CAPSTONE LAUNCH

You have a plan. You have a path. Let's begin.

1. APPROVED OR REVISING SCOPE

Your proposal is reviewed and your scope is clear.

- ☐ Purpose is clear
- ☐ Features are focused
- ☐ Statement fits the idea
- ☐ Risks and boundaries noted
- ☐ Scope boundary is defined
- ☐ Scope fits time and course

If revising:
Take the changes needed. Then continue the path forward.

2. FIRST USEFUL BUILD TARGET

You will build the smallest, workable core that proves the idea works.

Focus on:

- One simple feature
- One clear input
- One visible output
- One basic validation

Goal:
Create something useful, testable, and explainable.

3. EVIDENCE COLLECTION STARTED

Capture the evidence as you build. This shows your thinking and progress.

Collect as you go:

- Project Plan
- Build notes
- Sample input / output
- Validation notes
- Review notes
- All-in notes

Why it matters:
Others can see your work as it grows.

YOU'RE LAUNCHING, NOT FINISHING.

This is the beginning of your capstone journey. Build, test, learn, and improve.

PROGRESS OVER PERFECTION.

Small steps today lead to something you're proud of.

Start small. Stay explainable.

Success Check

40

Southwest
Tech



Thank you!

Have Fun!

41
